



Pre-Reads

14-Day AI Accelerator Program

Pre-Read V

Session Name:

Gen AI Beyond Prompting: Introduction to MCPs

Table of Contents

Chapter 1: Model Context Protocol (MCP)

- 1.1 What is MCP?
- 1.2 Why MCP matters for modular, persistent AI
- 1.3 MCP architecture: hosts, clients, servers, memory
- 1.4 Core MCP entities: tools, resources, prompts, context memory
- 1.5 Standardising tools and data access for any AI model
- 1.6 Context management: persistent memory, session state, and chaining
- 1.7 Security, compliance, and versioning in MCP workflows
- 1.8 Best practices for no-code builders using MCP
- 1.9 Advanced MCP use-cases: multi-agent orchestration, multi-modal context, and cross-platform integration

Chapter 2: Claude MCP – Persistent Context & Project-Driven Workflows

- 2.1 Injecting long-term memory into Claude workflows
- 2.2 Claude as a data analyst: project-based context retention
- 2.3 Claude + Blender: automated storyboard generation
- 2.4 Building and demoing Claude micro-apps (memory + interface patterns)

Chapter 3: Perplexity MCP – Threaded Reasoning & Research Automation

- 3.1 Multi-hop reasoning with persistent query chains
- 3.2 Prompt programming for evolving research workflows
- 3.3 Use case: competitive research assistant with voice integration

Chapter 4: Getting Files from the Local System

- 4.1 Uploading PDFs, CSVs, and other files into AI workflows
- 4.2 Chunking, parsing, and structuring local data for context-aware processing

Chapter 5: Creating Stylised Documents Using MCP

- 5.1 Prompt → Format → Audience → Voice → Output: end-to-end content automation
- 5.2 Use case: auto-generate branded newsletters or HR policy docs
- 5.3 Claude + Editor pattern: automated content QA and revision

Chapter 6: Building an MCP Server

- 6.1 What is an MCP server, and how does it work?
- 6.2 Integrating Claude, VAPI, and other agents
- 6.3 Designing lightweight, serverless MCP pipelines
- 6.4 Visual flow: Prompt → MCP Server → Claude + VAPI → Output
- 6.5 Use case: instruction chaining across text, voice, and visual modalities

Chapter 7: APIFY – Real-Time Data Scraping for Claude Workflows

- 7.1 Feeding live scraped data into Claude-powered projects
- 7.2 Orchestrating APIFY with MCP for dynamic, context-rich automation

Chapter 8: KITE – Workflow-Level Memory Switching and Context Management

8.1 How KITE enables seamless context switching in multi-project environments

8.2 Integrating KITE with MCP for persistent, adaptive AI agents

Chapter 9: VAPI MCP – Voice Agents with Context Retention

9.1 Building conversational interfaces and voice agents using VAPI and MCP

9.2 Context retention, Claude fallback, and multi-modal workflows

Chapter 1: Model Context Protocol (MCP) – Foundations & Advanced Patterns

1.1 What is MCP?

Imagine your AI is a brilliant but forgetful new employee. Every time you start a new conversation, they forget everything you've ever discussed, who your company is, and what tools they're supposed to use. It's frustrating and inefficient.

Model Context Protocol (MCP) solves this. Think of it as a **universal smart backpack for any AI model**. It's not an AI model itself, but a set of rules and connections that give your AI:

- A perfect, long-term memory.
- A universal toolkit that any AI can use.
- A filing cabinet for all its project documents.

In short, MCP is the standard that lets your AI remember who you are, what you're working on, and how to get things done, session after session.

1.2 Why MCP matters for modular, persistent AI

For no-code builders, MCP is a game-changer.

- **Modularity:** You can easily swap AI models without losing your setup. If a new, better model than Claude comes out tomorrow, you can "unplug" Claude and "plug in" the new

model. Because your tools and memory are managed by MCP, the new model instantly knows how to work.

- **Persistence:** This is the "perfect memory." An AI powered by MCP can build on past conversations, learn your company's style guide, and remember the results of a research project from last week. No more copy-pasting context into every prompt.

Efficiency: Stop repeating yourself. Define a process once, like "summarise a PDF and format it as an email", and any AI can execute it with a single click.

1.3 MCP architecture: hosts, clients, servers, memory

Let's break down the MCP world using a simple analogy: **building a custom piece of furniture in a shared workshop.**

- **Host:** This is the workshop itself, your no-code platform (e.g., Zapier, [Make.com](https://make.com), or a custom builder). It's where you design the workflow.
- **Client:** This is your expert craftsman, the AI model (e.g., Claude, Perplexity). They do the actual "thinking" work.
- **Server (MCP Server):** This is the workshop's central toolshed and project manager. It holds all the tools and project files. When you give an instruction, the project manager fetches the right tools (e.g., a calculator, a web scraper) and the right project files for your craftsman (the AI).
- **Memory:** This is the filing cabinet inside the toolshed. It contains all the notes, past project details, and style guides.

1.4 Core MCP entities: tools, resources, prompts, context memory

These are the building blocks you'll work with directly:

- **Tools:** A specific capability you give your AI, like `send_email`, `scrape_website`, or `calculate_revenue`. You define the tool once, and MCP lets any AI use it.

- **Resources:** These are the raw materials, the data. A Resource could be an uploaded PDF, a CSV file of sales data, or a snippet of text from a website.
- **Prompts:** These are your instructions. But in MCP, prompts become smarter. Instead of just "summarise this," you say, "Use the Summarizer_Tool on the Latest_Annual_Report.pdf Resource."

Context Memory: This is the AI's notebook. It's where key information is stored long-term. For example, after analysing a report, you can tell the AI: "Save the key findings to the Project_X_Memory." Next time, you can just ask, "What were the key findings for Project X?"

1.5 Standardising tools and data access for any AI model

Before MCP, giving an AI a tool was like building a custom power tool that only one person knew how to use. If you switched AIs, you had to build a whole new tool.

MCP acts like a **universal power adapter**. You create a standard "plug" for your tool (e.g., an API endpoint). Now, any MCP-compatible AI model knows exactly how to connect to it and use it. This means less setup and more flexibility. You can build a library of powerful tools and give access to any AI, creating an entire team of specialized agents.

1.6 Context management: persistent memory, session state, and chaining

An AI's memory isn't just one thing. MCP helps manage different types:

- **Persistent Memory:** This is long-term memory. It's for things the AI should *never* forget, like your company name, your brand's tone of voice, or your personal preferences.
- **Session State:** This is short-term memory, for the current conversation only. "Summarise the last three paragraphs." Once the conversation ends, this memory is cleared.
- **Chaining:** This is how you link conversations together to create a workflow. The output of one prompt becomes the input for the next. MCP manages this automatically, so the AI understands that the context is flowing from Step 1 to Step 2.

1.7 Security, compliance, and versioning in MCP workflows

When your AI has access to important data, security is crucial.

- **Security:** MCP allows you to set permissions. You can define which AIs or workflows can access specific Resources or Memory. For example, your "HR Policy Bot" can access HR documents, but your "Marketing Content Bot" cannot.
- **Compliance:** For industries like healthcare or finance, MCP helps create auditable trails. Every action taken by the AI, every tool used, and every piece of data accessed can be logged.
- **Versioning:** What if you update a prompt or a tool? Versioning lets you save different versions of your MCP assets. You can test a new prompt ("v2") while the old one ("v1") is still running, or roll back to a previous version if something breaks.

1.8 Best practices for no-code builders using MCP

- **Name Everything Clearly:** Don't call a resource doc1.pdf. Call it Q4_2024_Sales_Report.pdf. Your future self will thank you.
- **Start Small:** Automate one simple task first. Create one tool, one resource, and one prompt. See it work end-to-end.
- **Decouple Memory and Action:** Store your company's "brand voice" in Persistent Memory. Then, create separate prompts that *use* that memory to write a blog post, an email, or a tweet.
- **Think in Workflows:** Don't just write a giant prompt. Break your task into logical steps and chain them together using MCP.

1.9 Advanced MCP use-cases: multi-agent orchestration, multi-modal context, and cross-platform integration

This is where MCP truly shines.

- **Multi-Agent Orchestration:** Imagine an AI team. You give a high-level goal: "Create a marketing campaign for our new product." The MCP Server acts as the manager:

1. It assigns a Research_Agent (using Perplexity) to find market data.
 2. The results are saved to MCP memory.
 3. A Copywriter_Agent (using Claude) reads the memory and writes ad copy.
 4. A Voice_Agent (using VAPI) turns the copy into a radio ad script.
- **Multi-Modal Context:** Your context isn't just text anymore. It can be an image, a voice command, or a data table. MCP can store all these different types in one project context, allowing an AI to, for example, "describe this image in the style of our brand voice."
 - **Cross-Platform Integration:** Your workflow might start with a voice command on your phone (VAPI), trigger a process in a no-code tool (Zapier), use Claude for text generation, and finally post a message in Slack. MCP is the invisible thread that connects all these platforms, carrying the context seamlessly between them.

Chapter 2: Claude MCP – Persistent Context & Project-Driven Workflows

2.1 Injecting long-term memory into Claude workflows

By default, each chat with Claude is a fresh start. With MCP, you can give Claude a "project folder" in its brain. Before your prompt is sent to Claude, the MCP server automatically "injects" relevant context.

Example:

1. **Setup:** You create a Persistent Memory in MCP called MyBrandVoice and fill it with your company's style guide.
2. **Workflow:** When you prompt Claude, "Write a blog post about AI productivity," your MCP workflow is configured to automatically add: "Remember to use the MyBrandVoice guidelines: friendly, professional, and optimistic tone."

Claude now writes perfectly on-brand content every time, without you needing to remind him.

2.2 Claude as a data analyst: project-based context retention

Let's say you get a monthly sales report as a CSV file.

- **Without MCP:** Each month, you upload the new CSV and write a long prompt explaining what to look for and how to compare it to previous months.
- **With MCP:**
 1. **Month 1:** You upload sales_jan.csv as an MCP Resource. You ask Claude to analyse it and save the summary to a Persistent Memory called MonthlySalesSummaries.
 2. **Month 2:** You upload sales_feb.csv. Your prompt is simple: "Analyse the new sales data and compare it to the key findings in MonthlySalesSummaries." Claude, via MCP, accesses its memory of January's data and provides a comparative analysis instantly.

2.3 Claude + Blender: automated storyboard generation

This shows the power of connecting different types of tools. Blender is a 3D graphics program with an API.

1. **Prompt Claude:** "Write a simple three-scene storyboard for a commercial about coffee. A person wakes up tired, drinks coffee, and then feels energetic."
2. **MCP Server takes over:**
 - It sends the prompt to Claude.
 - Claude returns a structured description: Scene 1: Close up on face, yawning. Scene 2: Hand holding a steaming coffee mug. Scene 3: Person smiling, typing on a laptop.
 - The MCP Server formats this text into a command that Blender's API can understand.
 - It calls the Blender API, which generates three simple placeholder images. The result is an automated visual draft created from a simple text prompt.

2.4 Building and demoing Claude micro-apps (memory + interface patterns)

A micro-app is a simple, single-purpose tool you can build for your team.

Example: The "On-Brand Email Responder"

- **Backend (MCP):**
 - Memory: A persistent memory containing your company's brand voice and common customer FAQs.
 - Tool: A connection to your email service (e.g., Gmail API).
- **Frontend (a simple no-code UI builder):**
 - A text box labelled "Paste Customer Email Here."
 - A button labelled "Generate Draft Response."
- **How it works:** A team member pastes a customer's email and clicks the button. The text is sent to the MCP server, which prompts Claude: "Using our BrandVoice and FAQ_Memory, draft a reply to this customer email." The draft appears back in the UI, ready to be sent.

Chapter 3: Perplexity MCP – Threaded Reasoning & Research

Automation

3.1 Multi-hop reasoning with persistent query chains

Perplexity excels at research. MCP gives it a memory to connect its findings. "Multi-hop reasoning" means using the answer to one question to form the next question.

Example:

- **Prompt 1:** "Who are the main competitors for Slack?"
 - Perplexity answers: Microsoft Teams, Google Chat, etc.
 - MCP saves this list to Session Memory.
- **Prompt 2:** "For each of them, what is their primary pricing model?"
 - You don't have to list the competitors again. The MCP server takes the list from memory and automatically runs a new Perplexity query for each one. This creates a "chain" of reasoning.

3.2 Prompt programming for evolving research workflows

Your research process can be saved and reused.

Imagine you have a standard "Competitor Analysis" workflow. In MCP, you can build this as a sequence of prompts:

1. "Identify the top 5 competitors for [PRODUCT]."
2. "For each competitor, find their G2 rating."
3. " Summarise the most common negative feedback from their reviews."
4. "Put all this information into a markdown table."

You save this entire chain as a single MCP "workflow." Now, anytime you want to research a new product, you just run the workflow and fill in the [PRODUCT] blank.

3.3 Use case: competitive research assistant with voice integration

Let's combine Perplexity, MCP, and a voice tool like VAPI.

1. **You (speaking to your phone):** "Hey AI, start a new research project on the company 'Figma'."
2. **VAPI** transcribes your voice and sends the command to the **MCP Server**.
3. **MCP** creates a new context space for "Project Figma" and uses **Perplexity** to get a company overview, saving it to memory.
4. **Later, you ask:** "What were their last funding rounds?"
5. **VAPI + MCP** understand you're still talking about "Project Figma." It queries Perplexity for the funding info, retrieves it, and VAPI reads the answer back to you. The entire research history is saved and accessible.

Chapter 4: Getting Files from the Local System

4.1 Uploading PDFs, CSVs, and other files into AI workflows

The first step to working with your data is getting it into the system. MCP provides a standardised "inbox." Instead of uploading a file directly to Claude (and having it be forgotten),

you upload it to your MCP Resources. This gives it a permanent ID (e.g., Q1_Marketing_Report.pdf) that you can reference in any prompt, for any AI.

4.2 Chunking, parsing, and structuring local data for context-aware processing

AIIs have a limited "attention span" (context window). You can't just give Claude a 200-page book and expect it to understand everything. MCP automates the prep work.

- **Chunking:** An MCP utility can automatically break a long document into smaller, thematically related "chunks."
- **Parsing:** If you upload a CSV, MCP can automatically identify the columns and rows, turning it into a structured table that the AI can easily read.
This means the AI gets data that is already pre-digested and ready for analysis, leading to much better results.

Chapter 5: Creating Stylised Documents Using MCP

5.1 Prompt → Format → Audience → Voice → Output: end-to-end content automation

This is the "Content Factory" pattern. You create MCP Memory assets for each part of your style.

- Format: "A 5-paragraph blog post with a title, intro, three points, and conclusion."
- Audience: "Beginners in AI, no technical jargon."
- Voice: "Enthusiastic, helpful, and slightly humorous."

Your prompt becomes a simple instruction that assembles these pieces: *"Write about [TOPIC] using the Blog_Post_Format for a Beginner_Audience in our Enthusiastic_Voice."*

5.2 Use case: auto-generate branded newsletters or HR policy docs

- **Newsletter:** Your MCP workflow pulls the latest 3 articles from your blog (using a web-scraping Tool), then uses your Newsletter_Format and Brand_Voice to assemble a draft for Mailchimp.

- **HR Policy:** You prompt, "Create a 'Work From Home' policy doc." The MCP workflow fetches your HR_Policy_Template, your Formal_Legal_Voice, and generates a standardised, correctly formatted document.

5.3 Claude + Editor pattern: automated content QA and revision

A powerful workflow where one AI instance checks the work of another.

1. **Claude "The Writer":** Gets a prompt to write a first draft.
2. **MCP Server:** Saves the draft.
3. **Claude "The Editor":** A second prompt is triggered, but this one is different. It's given the draft and a checklist stored in MCP Memory (e.g., "Check for passive voice. Ensure tone matches Brand_Voice. Verify all claims are from the Source_Document.pdf."). It then revises the draft.

Chapter 6: Building an MCP Server

6.1 What is an MCP server, and how does it work?

Don't let the word "server" scare you. An MCP server isn't a physical box in your office. It's a configuration you set up in a cloud service (often with a no-code interface). It acts as the **central brain or traffic cop** for all your AI agents and tools. When a request comes in, the MCP server looks at the instruction, identifies the necessary tools and memory, and directs the work to the correct AI.

6.2 Integrating Claude, VAPI, and other agents

Your MCP server has a "connections" panel. Here, you plug in your different services. You'll add your Claude API key, your VAPI credentials, your Apify account, etc. This is like adding contacts to your phone. Once they're added, the MCP server knows how to "call" them.

6.3 Designing lightweight, serverless MCP pipelines

"Serverless" is a fantastic concept for non-coders. It means you design the workflow, but you don't have to worry about keeping a computer running 24/7. The infrastructure only "wakes up" when your workflow is triggered, runs the job, and goes back to sleep. You only pay for the exact moments it's working. This is how you can build incredibly powerful AI systems without the cost or complexity of traditional servers.

6.4 Visual flow: Prompt → MCP Server → Claude + VAPI → Output

Here's a simple visual journey of a request:

Your Voice Command → **VAPI** (Transcribes to text) → **MCP Server** (Receives text, adds context from Memory) → **Claude** (Processes text + context, generates a response) → **MCP Server** (Receives response) → **VAPI** (Converts text response to speech) → Audio Output to you

6.5 Use case: instruction chaining across text, voice, and visual modalities

"Scan this invoice (image), pay the amount listed using our payments tool, and send me a voice confirmation when it's done."

1. **Image:** An OCR Tool reads the invoice image.
2. **Text:** The extracted amount is sent to a payment Tool.
3. **Action:** The payment is processed.
4. **Voice:** VAPI sends a voice message to your phone: "The invoice for \$125.50 has been paid."

(Chapters 7-10 would continue this practical, use-case-driven approach, integrating specific (fictional or real) tools into the MCP ecosystem.)

Chapter 7: APIFY – Real-Time Data Scraping for Claude Workflows

- Explains how to use Apify as a Tool in MCP to give Claude up-to-the-minute data from any website, overcoming its knowledge cut-off.
- **Use Case:** A stock market watcher that scrapes news headlines and asks Claude to summarise market sentiment.

Chapter 8: KITE – Workflow-Level Memory Switching and Context Management

- Presents "KITE" as a user-friendly layer on top of MCP for managing multiple, distinct projects.
- **Analogy:** KITE is like switching between different user profiles on a computer. When you switch to "Project Marketing," it loads all the marketing-related memories, tools, and resources. When you switch to "Project Finance," it loads a completely different set.

Chapter 9: VAPI MCP – Voice Agents with Context Retention

- Focuses on how MCP's persistent memory transforms voice assistants from simple command-takers into true conversational partners.

Use Case: A voice-activated project manager. You can say, "What's on my plate for the 'Website Relaunch' project?" and it will pull the context from MCP and give you a relevant, up-to-date answer. If it doesn't know, it uses the Claude fallback to reason an answer from the project files.